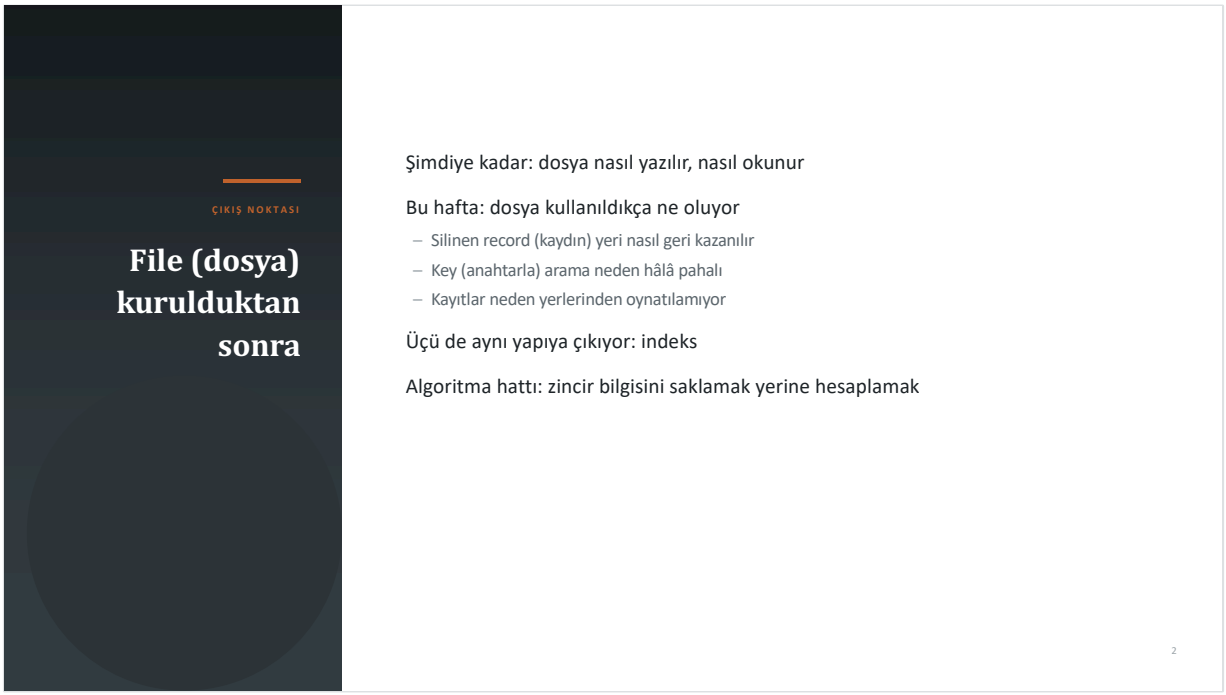
TRAKYA ÜNİVERSİTESİ · BİLGİSAYAR MÜHENDİSLİĞİ

Dosya Organizasyonu

Hafta 7 · Kayıt Erişimi ve Alan Geri Kazanımı

Doç. Dr. Emir Öztürk

2025/2026 Güz Yarıyılı

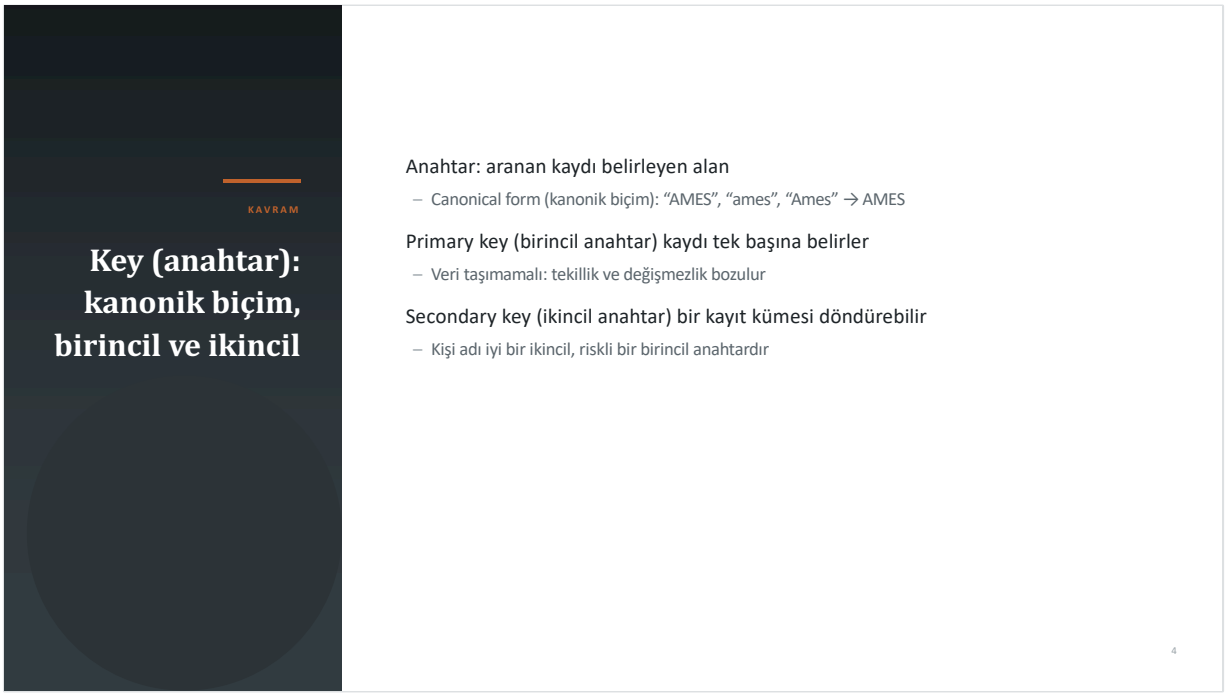


Yedinci haftanın çıkış noktası, file (dosyanın) kurulduktan sonra da yaşamayı sürdürmesidir. Önceki haftalarda dosyanın nasıl yazıldığı ve nasıl okunduğu incelenmiştir. Bu hafta dosyanın kullanım sürecinde uğradığı değişiklikler ele alınmaktadır: record (kayıt) eklenmesi, güncellenmesi ve silinmesi. Silinen kaydın bıraktığı yerin yeniden kullanılması, aramanın maliyeti ve kaydın yerinin değiştirilememesi bu haftanın üç ana sorunudur. Üçü birlikte indeks fikrine götürmektedir. Algoritma hattında computed chaining ele alınmaktadır. Yöntem, kayıtların zincir boyunca taşınmasına dayandığından haftanın silme ve pinned record (iğnelenmiş kayıt) tartışmalarıyla doğrudan ilişkilidir.

1

Anahtar türleri

Anahtar, sequential search (sıralı arama), doğrudan erişim



Aranan kaydı belirleyen alana key (anahtar) denir. Kullanıcı aynı değeri farklı biçimlerde yazabileceğinden arama öncesinde değer canonical form (kanonik biçim) adı verilen tek bir gösterime dönüştürülür; örneğin yalnızca büyük harf kullanılır ve sondaki boşluklar atılır. Primary key (birincil anahtar) kaydı tek başına belirler ve tekilliği ekleme sırasında sınanır. Birincil anahtarın veri taşımaması gerekir. Secondary key (ikincil anahtar) kaydı tek başına belirlemez ve bir kayıt kümesi döndürebilir.

2

Silme ve alanın geri kazanılması

Avail list

SORUN

Silme ve silinen kaydın işaretlenmesi

Ekleme dosyayı bozmaz; bozulma silmeyle başlar

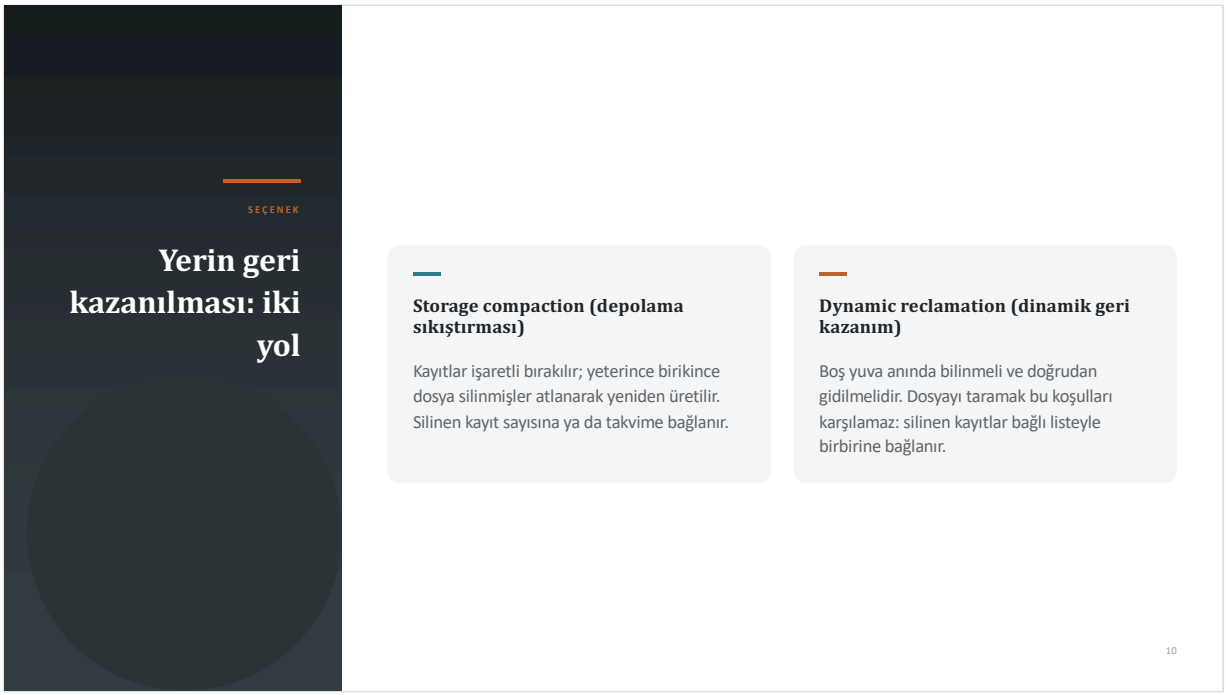
- Güncelleme = silme + ekleme: odak silmededir

Silinen kayıt tanınabilir olmalıdır

- İlk alana yıldız ya da ayrı bir silme alanı
- Veri bozulmaz, silme geri alınabilir

Dosyayı kullanan her program işaretli kayıtları atlar

Bir dosya üzerinde üç tür değişiklik yapılır: ekleme, güncelleme ve silme. Yalnızca ekleme yapılan dosyada bozulma olmaz. Bozulma, kayıt silindiğinde ve değişken uzunluklu bir kayıt güncellendiğinde başlar; güncelleme de silme ile eklemenin birleşimi sayılabileceğinden odak silmededir. Silinen kaydın tanınabilir olması gerekir. Ayraçlı yapıda ilk alana bir yıldız konabilir, ancak işaretin ayrı bir alanda tutulması daha iyidir: veri bozulmaz ve silme işlemi geri alınabilir. Dosyayı kullanan her programın işaretli kayıtları atlaması gerekir.



Silinen kaydın yeri iki yolla geri kazanılabilir. Storage compaction (depolama sıkıştırması) yerin hemen kazanılmasını gerektirmez: kayıtlar bir süre işaretli bırakılır, yeterince biriktiğinde dosya silinmiş kayıtlar atlanarak yeniden üretilir. Bu işin ne zaman yapılacağı silinen kayıt sayısına ya da takvime bağlanabilir; muhasebede dönem sonu yaygındır. Etkileşimli uygulamalar ise bu işlemi bekleyemez. Yerin hemen yeniden kullanılabilmesi için boş bir yuvanın var olup olmadığı anında bilinmeli ve oraya doğrudan gidilmelidir. Dosyayı baştan sona taramak iki koşulu da karşılamadığından silinen kayıtlar bir bağlı listeyle birbirine bağlanır.

YAPI

Avail list

Silinen kayıtlardan oluşan bağlı liste

Yığın olarak işletilir

- Yeni silinen kayıt listenin başına konur
- Yer gerektiğinde baştaki yuva alınır

Sabit uzunlukta bu seçim yeterlidir

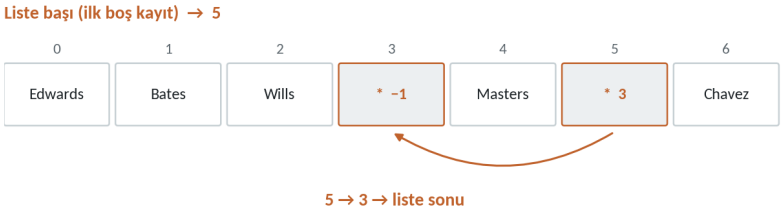
- Bütün yuvalar aynı boydadır, hiçbiri yeğlenmez

Listenin başı dosyanın header record (başlık kaydında) tutulur

İki koşul, silinen kayıtların bağlı listeyle birbirine bağlanmasıyla karşılanır. Silinen kayıtlardan oluşan bu listeye avail list denir. Liste yığın olarak işletilir: yeni silinen kayıt listenin başına konur, yer gerektiğinde baştaki kayıt alınır. Fixed-length record (sabit uzunluklu kayıtlarda) bu seçim yeterlidir, çünkü bütün yuvalar aynı boydadır ve hiçbiri diğerine yeğlenmez. Listenin başı dosyanın header record (başlık kaydında) tutulur.

ÖRNEK

3 ve 5 numaralı kayıtların silinmesi



Silinen kayıt yerinde kalır; ikinci alanı bir sonraki boş kaydın RRN'sini taşır

Kayıtlar taşınmadı, yalnızca bağlantılar kuruldu

Yeni kayıt eklendiğinde listenin başındaki yuva kullanılır

12

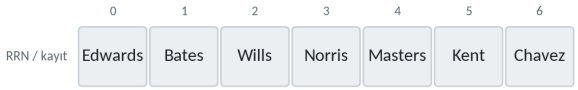
Şekilde yedi kayıtlık bir dosyada üç ve beş numaralı kayıtların bu sırayla silinmesinin sonucu görülmektedir. Beş numaralı kayıt listenin başındadır, çünkü en son silinen odur. Onun link field (bağlantı alanı) üç numarayı gösterir, üç numaranınki ise liste sonu işaretini taşır. Silinen kayıtlar hiçbir yere taşınmamıştır. Yalnızca bağlantılar kurulmuştur. Yeni kayıt eklendiğinde listenin başındaki yuva kullanılır ve liste başı bir sonrakine kayar.

ADIM 0 / 5

Avail list

Başlangıç: yedi kayıt

başlık: liste başı = −1



Liste boş: başlıkta −1

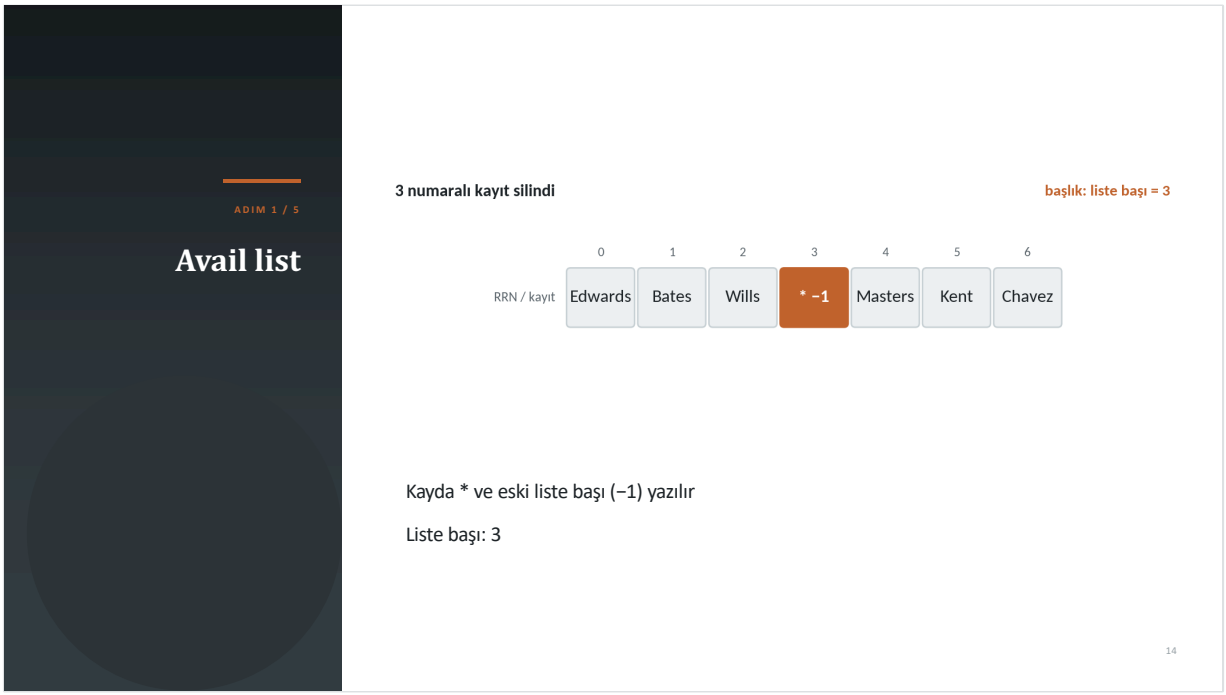
Yeni kayıt dosyanın sonuna eklenir

13

Başlangıçta dosyada sıfırdan altıya numaralanmış yedi kayıt bulunmaktadır ve hiçbir kayıt silinmemiştir. Header record (başlık kaydındaki) liste başı alanı, liste boş olduğunu belirten eksi bir değerini taşımaktadır. Bu durumda yer isteyen işlev, yeni kaydı dosyanın sonuna, yedi numaralı konuma yerleştirecektir.

ADIM 1 / 5

Avail list



Üç numaralı kayıt silinmektedir. Kaydın başına silme işareti, ardına ise o ana kadarki liste başı olan eksi bir yazılmaktadır. Başlıktaki liste başı üç olarak güncellenmektedir. Kayıt yerinden kımıldamamış, yalnızca iki alanı değişmiştir. Liste şimdi tek elemanlıdır.

ADIM 2 / 5

Avail list

5 numaralı kayıt silindi

başlık: liste başı = 5



Kayda * ve eski liste başı (3) yazılır

Liste: 5 → 3 → −1

15

Beş numaralı kayıt silinmektedir. Silinen kayda, o ana kadarki liste başı olan üç değeri bağlantı olarak yazılmakta ve liste başı beş olmaktadır. Liste yığın olarak işletildiğinden en son silinen kayıt listenin başındadır. Liste artık beş, üç ve liste sonu biçiminde izlenmektedir.

ADIM 3 / 5

Avail list

Yilmaz eklendi

başlık: liste başı = 3



Liste başındaki 5 kullanılır

Liste başı bir sonrakine kayar: 3

16

Yeni bir kayıt eklenmek istenmektedir. Yer isteyen işlev listenin başındaki beş numaralı yuvayı döndürmekte, yuvanın link field (bağlantı alanındaki) üç değerini okuyarak liste başına yazmaktadır. Yeni kayıt beş numaralı konuma yerleşmektedir. Dosyanın sonuna ekleme yapılmadığından dosya büyümemiştir.

ADIM 4 / 5

Avail list

Kaya eklendi

başlık: liste başı = −1



Liste başındaki 3 kullanılır

Liste yeniden boş: −1

17

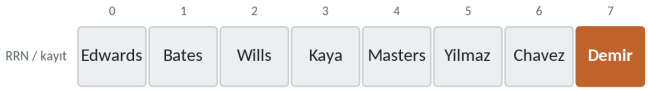
İkinci eklemede liste başındaki üç numaralı yuva kullanılmaktadır. Yuvanın link field (bağlantı alanı) liste sonu işaretini taşıdığından liste başı yeniden eksi bir olmaktadır. Silinen iki kaydın yeri böylece dosya taranmadan, her biri tek erişimle geri kazanılmıştır. Yuvaların silinme sırasının tersine kullanılması, listenin yığın olarak işletilmesinin sonucudur.

ADIM 5 / 5

Avail list

Demir eklendi

başlık: liste başı = −1



Liste boş: kayıt dosyanın sonuna

RRN 7, kayıt sayısı 8

18

Üçüncü eklemede liste boştur. Yer isteyen işlev bu kez dosyanın sonundaki yedi numarasını döndürmekte ve kayıt sayısını bir artırmaktadır. Yeni kayıt yedi numaralı konuma yazılmaktadır. Aynı sıra, izleyen kod örneğinde programın çıktısıyla doğrulanmaktadır. Header record (başlık kaydındaki) kayıt sayısı alanı, sona eklenecek kaydın numarasını bu nedenle her zaman hazır tutmaktadır.

3

Variable-length record (değişken uzunluklu kayıtların) silinmesi

Doğru boyda yuva bulmak

GEREKSINIM

Değişken uzunlukta yeniden kullanım

Üç şey gerekir: bağlantı alanı, listeye ekleme, listeden uygun yuvayı alma

Bağlantı kayıt numarası değil bayt uzaklığı taşır

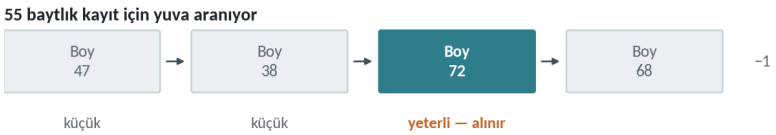
- Kayıt numarasından uzaklık hesaplanamaz
- Kaydın başına yıldız, ardına sonraki yuvanın uzaklığı

Kaydın sınırı başındaki bayt sayısıyla belirlidir

Silinen kayıtların yeniden kullanılması için üç şey gereklidir: silinen kayıtları birbirine bağlayacak bir alan, yeni silinen kaydı listeye ekleyen bir algoritma ve listeden uygun kaydı bulup çıkaran bir algoritma. Variable-length record (değişken uzunluklu kayıt) bu yapıya elverişlidir, çünkü kaydın sınırı başındaki bayt sayısıyla belirlidir. Sabit uzunlukta bağlantı bir kayıt numarası taşıyordu. Değişken uzunlukta kayıt numarasından bayt uzaklığı hesaplanamadığından bağlantı uzaklığın kendisini taşır: kaydın ilk alanına yıldız, ardına bir sonraki boş kaydın uzaklığı yazılır. Bunun dışında yapı sabit uzunluktakiyle aynıdır.

ARAMA

Doğru boyda yuva



Yuva listeden çıkarılır: önceki kaydın bağlantısı onu atlayacak biçimde kurulur

- Yığından ilk yuvayı almak artık yetmez
- Liste, yeterli büyüklükte yuva bulunana dek dolaşılır
- Bulunamazsa kayıt dosyanın sonuna eklenir

Yığından ilk yuvayı almak artık yeterli değildir. Yuvanın yeni kaydı alacak kadar büyük olması gerekir. Listede, yeterli büyüklükte bir yuva bulunana dek ilerlenir. Şekildeki örnekte elli beş baytlık bir kayıt için sırasıyla kırk yedi, otuz sekiz ve yetmiş iki baytlık yuvalar incelenmektedir. İlk ikisi küçük geldiğinden üçüncüsü alınmaktadır. Listenin sonuna gelindiğinde uygun yuva bulunamazsa kayıt dosyanın sonuna eklenir.

İŞLEM

Listeden çıkarma ve ekleme

Alınan yuva listeden çıkarılır

- Bir önceki kaydın bağlantısı yuvayı atlayacak biçimde kurulur

Yeni silinen kayıt listenin başına konabilir

- Alma yordamı gerektiğinde listenin tamamını dolaşıyor
- Uygun yuva listede varsa arama onu eninde sonunda bulur

Bu nedenle ekleme için karmaşık bir yönteme gerek yoktur

Alınan yuva listeden çıkarılır: bir önceki kaydın bağlantısı, çıkarılan yuvayı atlayacak biçimde yeniden kurulur. Alma yordamı gerektiğinde listenin tamamını dolaştığı için, yeni silinen kayıtları listeye yerleştirirken karmaşık bir yönteme gerek yoktur. Uygun boyda bir yuva listede bir yerde bulunuyorsa arama onu eninde sonunda bulur. Bu nedenle yeni kayıtlar sabit uzunluklu durumdaki gibi listenin başına konabilir.

4

Fragmentation ve placement strategies

Parçalanma ve yerleştirme stratejileri: ilk uyan, en iyi uyan, en kötü uyan

KAVRAM

Internal fragmentation (iç parçalanma)



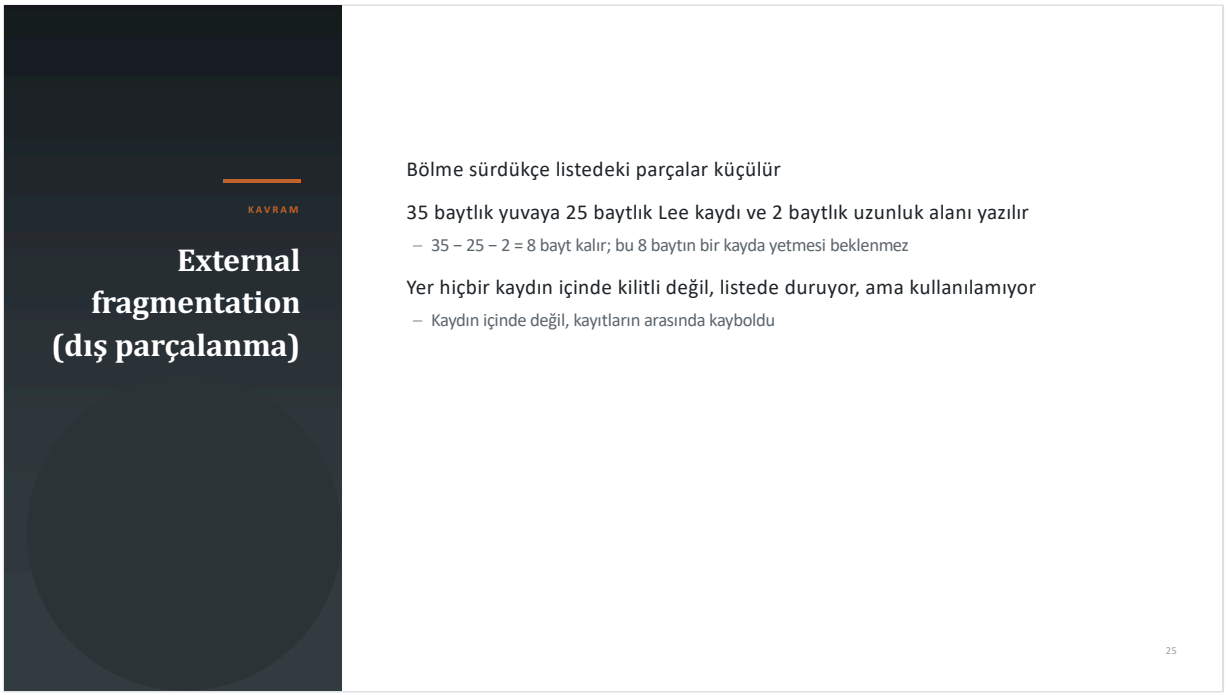
Kaydın içinde kalan kullanılmayan yer

- Ham kaydı 64 baytlık yuvada 37 baytı kilitler

Önlem: yuvayı bölmek; yeni parça 2 baytlık uzunluk alanı ister: 64 − 27 − 2 = 35

24

Internal fragmentation (iç parçalanma), kaydın içinde kalan kullanılmayan yerdir. Sabit uzunluklu kayıtlarda son alandan kaydın sonuna kadar uzanan dolgu bu türdendir. Değişken uzunluklu dosyada da oluşabilir: altmış dört baytlık bir yuvaya yirmi yedi baytlık Ham kaydı yazıldığında artan otuz yedi bayt kaydın parçası sayılır ve kullanılamaz. Önlem yuvayı bölmektir. Ancak bölünen her parça kendi uzunluk alanını taşımak zorundadır. Ham kaydının önüne iki baytlık yeni bir uzunluk alanı yazıldığından listeye dönen yer otuz yedi değil, altmış dört eksi yirmi yedi eksi iki, yani otuz beş bayttır.



Bölme işlemi sürdürüldükçe listedeki parçalar küçülür. Otuz beş baytlık yuvaya yirmi beş baytlık Lee kaydı yazıldığında, kaydın kendisinin yanında iki baytlık uzunluk alanı için de yer ayrılır. Bu nedenle geriye on değil, sekiz bayt kalmaktadır. Bu sekiz baytın bir kayda yetmesi beklenmez. Yer hiçbir kaydın içinde kilitli değildir ve avail list üzerinde durmaktadır, ancak hiçbir isteği karşılayamayacak kadar küçüktür. Buna external fragmentation (dış parçalanma) denir.

KARŞILAŞTIRMA

Internal ve external fragmentation (iç ve dış parçalanma)

	Internal (iç)	External (dış)
Yer nerede	kaydın içinde, yuvanın artanı	kayıtların arasında, avail list'te
Liste görür mü	hayır: yer kayda ait sayılır	evet: ama parça çok küçük
Nasıl oluşur	yuva bölünmezse	yuva bölündükçe
Ne zaman geri gelir	kayıt silindiğinde	birleştirme ya da sıkıştırmayla

Örnek: Ham kaydının içindeki 37 bayt iç, Lee'den artan 8 bayt dış parçalanmadır

Yuvayı bölmek iç parçalanmayı giderir, dış parçalanmayı doğurur

İki parçalanma türü yitirilen yerin nerede durduğuyla ayrılır. Internal fragmentation (iç parçalanmada) yer bir kaydın yuvası içindedir. Avail list bu yeri görmez, çünkü yer kayda ait sayılır; kayıt silinene dek geri kazanılamaz. External fragmentation (dış parçalanmada) yer hiçbir kayda ait değildir ve listede durur, ancak parçalar hiçbir isteği karşılayamayacak kadar küçüktür. Ham kaydının yuvasında kalan otuz yedi bayt birinci türe, Lee kaydından sonra kalan sekiz bayt ikinci türe örnektir. İkisi arasında bir ödünleşim vardır: yuvayı bölmek iç parçalanmayı giderirken dış parçalanmayı doğurur.

ÖNLEM

External fragmentation (dış parçalanmaya) karşı üç yol

Yol	Ne yapar	Ne zaman
Depolama sıkıştırması	dosyayı yeniden üretir	parçalanma katlanılmaz olunca
Delikleri birleştirme	bitişik iki yuvayı tek yuva yapar	sürekli
Yerleştirme stratejisi	parçalanmayı oluşmadan azaltır	her tahsis

Delikleri birleştirmenin zorluğu: avail list fiziksel sırada tutulmaz

- Bitişik iki yuvanın listede yan yana olması gerekmez

Üçüncü yol ayrı bir tartışmayı hak eder

STRATEJI 1

İlk uyan (first fit)

- Yeni silinen kayıtlar listenin başına konur
- Yer arandığında listenin başından başlanır
  - Yeni kaydı alacak ilk yuvada durulur
- Yuvanın gereğinden on kat büyük olması gözetilmez
- Listeye ekleme en az işi gerektiren yoldur

İlk uyan stratejisinde yeni silinen kayıtlar listenin başına konur ve yer arandığında listenin başından başlanarak yeni kaydı alacak ilk yuvada durulur. Yuvanın gereğinden on kat büyük ya da tam uyumlu olması gözetilmez. Listeye ekleme en az işi gerektiren yoldur. Arama ise ortalama olarak listenin bir bölümünü dolaşır. Ekleme sabit sürede yapılmakta, arama ise uygun yuva listenin başına yakınsa kısa sürmektedir.

STRATEJI 2

En iyi uyan (best fit)

Liste boya göre artan sırada: ilk yeterli yuva en küçüğüdür

- Ekleme sırayı korumalı, arama listenin bir bölümünü dolaşır

Sakınca: artan yer olabildiğince küçük kalır

- Kullanılamayan küçük yuvalar listenin başında birikir
- External fragmentation (dış parçalanmayı) besler

En iyi uyan stratejisinde liste boya göre artan sırada tutulur. Böylece karşılaşılan ilk yeterli yuva, işi görecek en küçük yuva olur ve uyum olabildiğince yakındır. Silinen kayıt listeye sırası korunarak eklenmeli, arama sırasında listenin bir bölümü dolaşılmalıdır. Daha önemli sakınca, kayıt yerleştikten sonra artan yerin çok küçük kalmasıdır. Bu yer çoğu zaman hiçbir kayda yetmez ve external fragmentation (dış parçalanmayı) besler. İşe yaramayan küçük yuvalar listenin başına yerleştiğinden sonraki aramalar da zamanla uzar.

STRATEJI 3

En kötü uyan (worst fit)

Liste boya göre azalan sırada tutulur

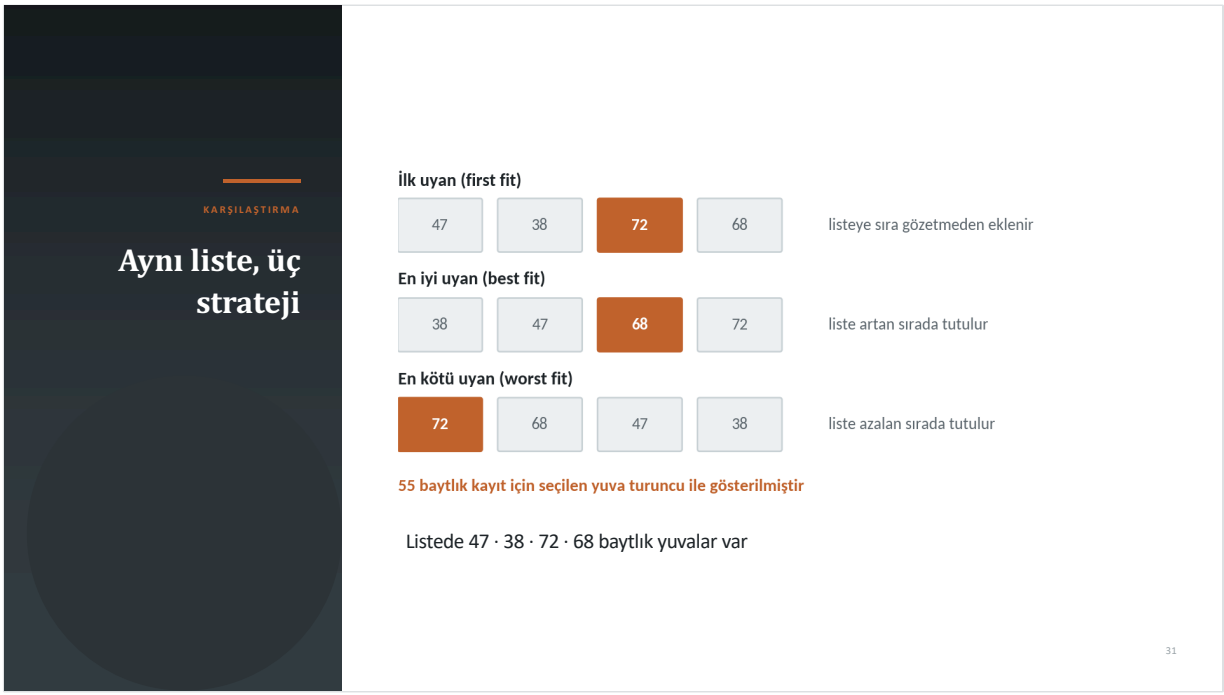
En büyük yuva her zaman listenin başındadır

- Alma yordamı yalnızca ilk öğeye bakar
- O yetmiyorsa hiçbiri yetmez

Yer en büyük yuvadan çıkarılır

- Artan bölüm olabildiğince büyük kalır
- Kullanılabilir olma olasılığı yükselir

En kötü uyan stratejisinde liste boya göre azalan sırada tutulur. En büyük yuva her zaman listenin başındadır. Alma yordamı yalnızca ilk öğeye bakar: o yetmiyorsa hiçbiri yetmez. Ayrıca yer en büyük yuvadan çıkarıldığı için artan bölüm olabildiğince büyük kalır ve kullanılabilir olma olasılığı yükselir. İlk bakışta akla yatkın görünmeyen strateji, external fragmentation (dış parçalanmanın) baskın olduğu durumda uygundur.



Şekilde 47, 38, 72 ve 68 baytlık dört yuvadan oluşan bir listeye 55 baytlık bir kayıt yerleştirilmektedir. İlk uyan stratejisinde liste silinme sırasıyla dolaşılmakta, 47 ve 38 baytlık yuvalar küçük geldiğinden 72 baytlık yuva seçilmekte ve 17 bayt artmaktadır. En iyi uyan stratejisinde liste artan sırada tutulduğundan 38 ve 47 baytlık yuvalar geçildikten sonra 68 baytlık yuva seçilmekte ve 13 bayt artmaktadır. En kötü uyan stratejisinde listenin başındaki 72 baytlık yuva tek bakışta seçilmekte, artan 17 bayt ise yeni bir kayda yetebilecek büyüklükte kalmaktadır. Aynı istek, stratejiye göre farklı bir yuvaya ve farklı büyüklükte bir artığa yol açmaktadır.

SEÇİM

Hangi strateji ne zaman

Durum	Uygun strateji
Sabit uzunluklu kayıtlar	**hiçbiri: yuvalar denk**
Yer iç parçalanmada yitiriliyor	**ilk uyan ya da en iyi uyan**
Yer dış parçalanmada yitiriliyor	**en kötü uyan**
Sık değişen, değişken uzunluklu dosya	**strateji anlamlı**

Hiçbir strateji her durumda üstün değildir

Yapılabilecek olan birkaç gözlem belirleyip duruma göre seçmektir

5

Aramanın sınırları ve keysort

İkili aramanın bedeli, kayıtları taşımadan sıralama

FIKIR

Yalnızca anahtarları sırala

Bellekte sıralarken gerçekte yalnızca anahtarlar sıralanır

Dosyanın tamamını belleğe almaya gerek yok

- Her kayıt geçici bir buffer (tampona) okunur
- Anahtarı alınır, kayıt atılır

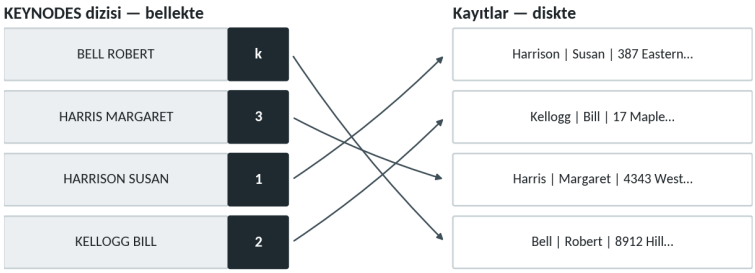
Bellekte yalnızca anahtar + kayıt numarası çiftleri tutulur

Aynı bellekle iç sıralamadan daha büyük dosya sıralanabilir

Keysort, bellekte sıralama yapılırken aslında yalnızca anahtarların sıralanması gerektiği gözlemine dayanır. Bu nedenle dosyanın tamamı belleğe alınmaz. Her kayıt geçici bir buffer (tampona) okunur, anahtarı alınır ve kayıt atılır. Bellekte yalnızca anahtar ile relative record number (göreli kayıt numarası) çiftleri tutulur. Keysort dosyanın tamamını belleğe almadığı için, aynı bellekle iç sıralamadan daha büyük dosyaları sıralayabilir.

YAPI

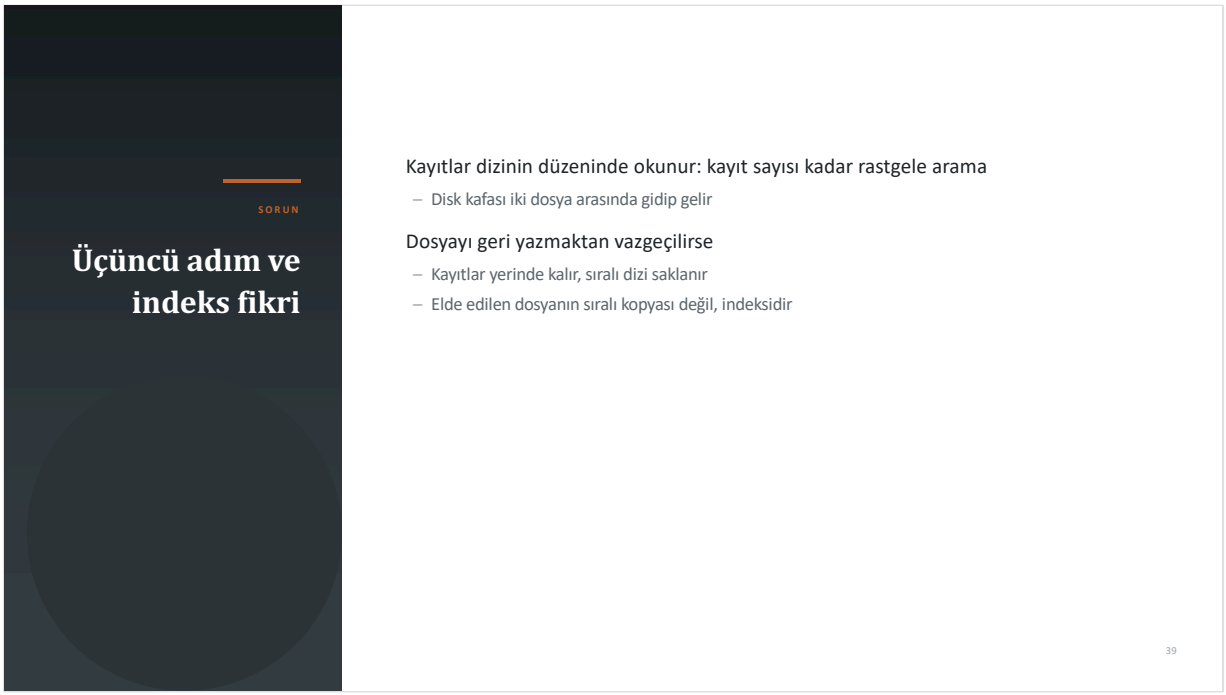
KEYNODES dizisi



Yalnızca anahtarlar sıralanır; kayıtlar yerinde durur

Sıralama bittiğinde directory (dizinin) ilk öğesi, dosyada ilk sıraya gelmesi gereken kaydı gösterir

Şekilde, bellekteki anahtar dizisi ile diskteki kayıtlar arasındaki ilişki görülmektedir. Directory (dizinin) her öğesi bir anahtar ile o anahtarın ait olduğu kaydın numarasını taşır. Sıralama yalnızca bu dizi üzerinde yapılır. Kayıtlar diskte yerinden kımıldamaz. Sıralama bittiğinde dizinin birinci öğesi, dosyada birinci sıraya gelmesi gereken kaydın numarasını gösterir. Kayıt numaraları birden başlatılmış, son kaydın numarası k ile gösterilmiştir. Dizinin her öğesi yalnızca bir anahtar ve bir numara taşıdığından, bir kaydın boyuna göre çok daha küçük yer tutar.



Keysort'un üçüncü adımında kayıtlar giriş dosyasından sıralı değil, dizinin düzeninde okunur. Bu, kayıt sayısı kadar rastgele arama demektir; okumalar çıktıya yazmalarla sırayla yapıldığından disk kafası iki dosya arasında gidip gelir. Bellekte sıralayarak kazanılan, dosyayı yeniden yazarken yitirilmektedir. Darboğaz yaratan bu adımdan vazgeçilebilir: sıralanmış dizi dosyaya yazılır, kayıtlar ise yerlerinde bırakılır. Elde edilen şey dosyanın sıralı bir kopyası değil, ona ait bir indekstir.

ÇÖZÜM

İndeks dosyası



Dosyayı sıralı yazmak yerine sıralı bir indeks üretilir — kayıtlar hiç taşınmaz

Önce indekste binary search (ikili arama), sonra RRN ile veri dosyasına tek erişim

İndeks dosyası, anahtar ile relative record number (göreli kayıt numarası) çiftlerinden oluşur ve anahtara göre sıralıdır. Aranan kayıt iki adımda bulunur: önce indeks dosyasında binary search (ikili arama) yapılır, sonra bulunan numarayla veri dosyasına tek erişimde gidilir. Veri dosyasındaki kayıtlar hiç taşınmamıştır. İndeks veri dosyasından çok daha küçük olduğu için ikili arama da daha az erişim gerektirir. İndeksin belleğe sığması durumunda disk erişimi bire iner.

6

Computed chaining

Zinciri saklamak yerine hesaplamak

NEDEN

Link field (bağlantı alanının) bedeli

Coalesced hashing (birleştirilmiş hashing): her kayda gerçek adres tutan bir alan

- Tablo büyüdükçe alan büyür: $\log_2$(tablo boyu) bit

Bağlantı kullanmayan yöntemler yer kazandırır, aramayı uzatır

Computed chaining: alanda adres değil küçük bir çarpan

- Sonraki adres = adres + çarpan × artış(anahtar)

Zincirle arama hızı korunur, alan zincir uzunluğu kadar küçülür

Coalesced hashing (birleştirilmiş hashing'in) başarımı link field (bağlantı alanından) gelir, ancak bu alan her kayda gerçek bir adres ekler ve tablo büyüdükçe alanın bit sayısı da büyür. Bağlantı kullanmayan yöntemler bu yeri harcamaz, buna karşılık aramayı uzatır. Computed chaining iki yaklaşımı birleştirir. Bağlantı alanında adres yerine küçük bir çarpan saklanır ve bir sonraki adres, bulunulan adrese bu çarpanın kayıttaki anahtardan hesaplanan artışla çarpımı eklenerek bulunur. Arama yine yalnızca kendi zinciri boyunca ilerler, alanın boyu ise tablo boyuna değil zincir uzunluğuna bağlı kalır.

GEREKÇE

Yer ile başarım arasındaki ödünleşim

Yaklaşım	Yer	Başarım
Bağlantı kullanan yöntemler	fazladan bağlantı alanı	yüksek
Bağlantı kullanmayan yöntemler	az	düşük
Computed chaining	az (yalnızca çarpan)	yüksek

İki yaklaşım bir arada kullanılarak hem başarım hem tasarruf sağlanıyor

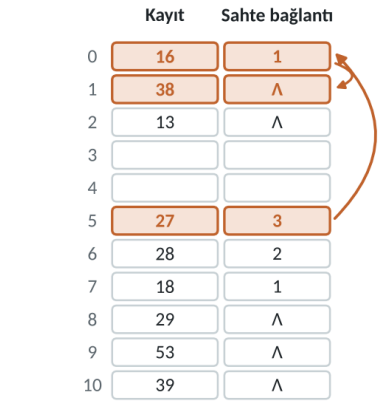
Link field (bağlantı alanı) adres yerine çarpan tutar: alan küçülür

Bağlantı kullanan yöntemlerin başarımı yüksektir, ancak link field (bağlantı alanı) için fazladan yere gerek duyarlar. Bağlantı kullanmayan yöntemler daha az yere gerek duyar, buna karşılık başarımları düşüktür. İki yaklaşımın bir arada kullanılmasıyla hem başarım artışı hem yerden tasarruf sağlanabilir. Computed chaining yöntemi bu amaçla geliştirilmiştir. Bağlantı alanında adres yerine küçük bir çarpan tutulması, alanın boyunu tablo boyunun logaritmasından zincir uzunluğunun logaritmasına indirmektedir.

YAPI

Pseudolink (sahte bağlantı) alanı

Bağlantı alanında adres değil, artış çarpanı tutulur



27 → 16 → 38

5 + 3×f(27) = 11 mod 11 = 0

0 + 1×f(16) = 1

sonraki adres = geçerli adres + çarpan × f(geçerli kayıt) · H(k) = k mod 11, f(k) = bölüm(k / 11)

Yöntemde her kayıt için sahte bir link field (bağlantı alanı) bulunur. Bu alanda, collision (çarpışma) durumunda kaydı yerleştirmek için bir sonraki kontrol edilecek adresi hesaplayan bir değer tutulur. Böylelikle az sayıda bit kullanılarak gerçek adresler elde edilebilmektedir. Bir sonraki adres, geçerli adrese o adresteki kaydın artış fonksiyonunun çarpan katı eklenerek bulunur. Alanda adres değil çarpan saklandığı için gereken bit sayısı belirgin biçimde azalır.

ALGORITMA

Kayıt ekleme

Anahtara hash function (hash fonksiyonunu) uygula

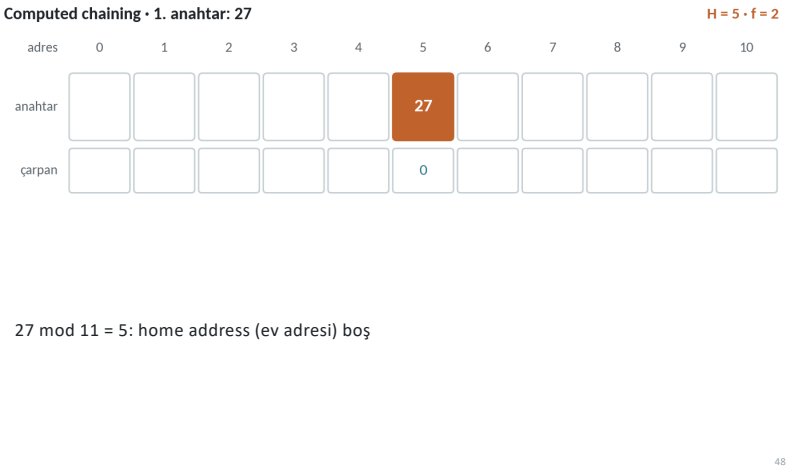
Home address (ev adresi) boşsa kaydı buraya ekle

Adres doluysa iki durum

- Depolanmış kayıt kendi ev adresindeyse: yeni kaydı zincirin sonuna yaz
- Boş yer ararken zincirdeki son kaydın artışını kullan: son + 1×f, son + 2×f, …
- Boş adresi pseudolink (sahte bağlantıyla) önceki kayda bağla
- Depolanmış kayıt ev adresinde değilse: o kaydı zincirde ilerlet
- Öncekinin artış fonksiyonunu kullanarak sonraki boş adrese taşı

ADIM 1 / 9

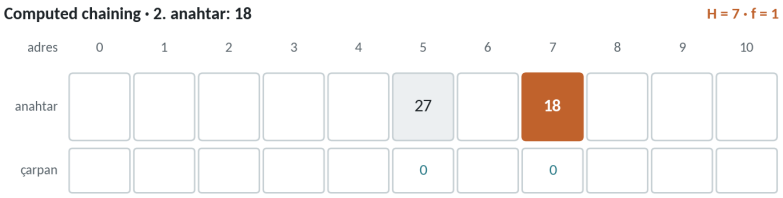
Computed chaining: 27



Birinci adımda 27 anahtarı eklenmektedir. Home address (ev adresi) 5, artış değeri 2'dir. Ev adresi boş olduğundan kayıt doğrudan yerleşmekte ve pseudolink (sahte bağlantı) alanına sıfır yazılmaktadır. Sıfır, zincirin burada bittiğini göstermektedir. Bu adımdan sonra tabloda bir kayıt bulunmaktadır.

ADIM 2 / 9

Computed chaining: 18



18 mod 11 = 7: home address (ev adresi) boş

49

İkinci adımda 18 anahtarı eklenmektedir. Home address (ev adresi) 7, artış değeri 1'dir. Ev adresi boş olduğundan kayıt doğrudan yerleşmekte ve pseudolink (sahte bağlantı) alanına sıfır yazılmaktadır. Sıfır, zincirin burada bittiğini göstermektedir. Bu adımdan sonra tabloda iki kayıt bulunmaktadır.

ADIM 3 / 9

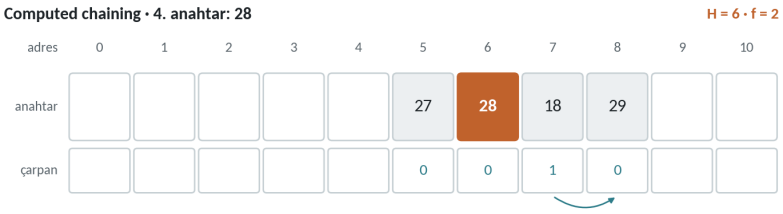
Computed chaining: 29



Üçüncü adımda 29 anahtarı eklenmektedir. Home address (ev adresi) 7'dir. Ev adresi doludur ve oradaki 18 kaydı kendi ev adresindedir; yeni kayıt zincirin sonuna eklenecektir. Zincirin son kaydı 7 adresindeki 18'dir. Boş yer aranırken 29'un değil, zincir sonundaki kaydın artış değeri kullanılır: f(18) = 1. Çarpan birden başlatılır ve 7 + 1×1 = 8 adresi boş bulunur. Son kaydın pseudolink (sahte bağlantı) alanına adres yerine 1 çarpanı yazılmaktadır.

ADIM 4 / 9

Computed chaining: 28



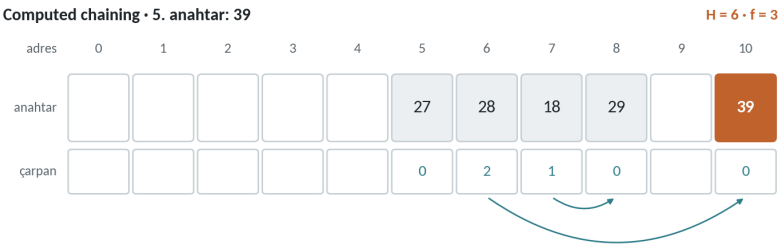
28 mod 11 = 6: home address (ev adresi) boş

51

Dörtüncü adımda 28 anahtarı eklenmektedir. Home address (ev adresi) 6, artış değeri 2'dir. Ev adresi boş olduğundan kayıt doğrudan yerleşmekte ve pseudolink (sahte bağlantı) alanına sıfır yazılmaktadır. Sıfır, zincirin burada bittiğini göstermektedir. Bu adımdan sonra tabloda dört kayıt bulunmaktadır.

ADIM 5 / 9

Computed chaining: 39



Home address (ev adresi) 6 dolu, yerleşik kayıt (28) kendi evinde

Artış zincir sonundaki kaydınki: f(28) = 2; 39'un artışı (3) kullanılmaz

Çarpan 1: 6 + 1×2 = 8 dolu (29) · çarpan 2: 6 + 2×2 = 10 boş

Bağlantıya 2 yazılır: arama bu değeri 28 üzerinden çözer

52

Beşinci adımda 39 anahtarı eklenmektedir. Home address (ev adresi) 6'dır. Ev adresi doludur ve oradaki 28 kaydı kendi ev adresindedir; yeni kayıt zincirin sonuna eklenecektir. Boş yer aranırken 39'un artış değeri olan 3 değil, zincir sonundaki 28'in artış değeri olan 2 kullanılır. Çarpan birden başlatılır. 6 + 1×2 = 8 adresi 29 ile doludur, 6 + 2×2 = 10 adresi boştur. Bu nedenle 39 dokuza değil ona yerleşir ve 28'in bağlantı alanına 2 yazılır.

ADIM 6 / 9

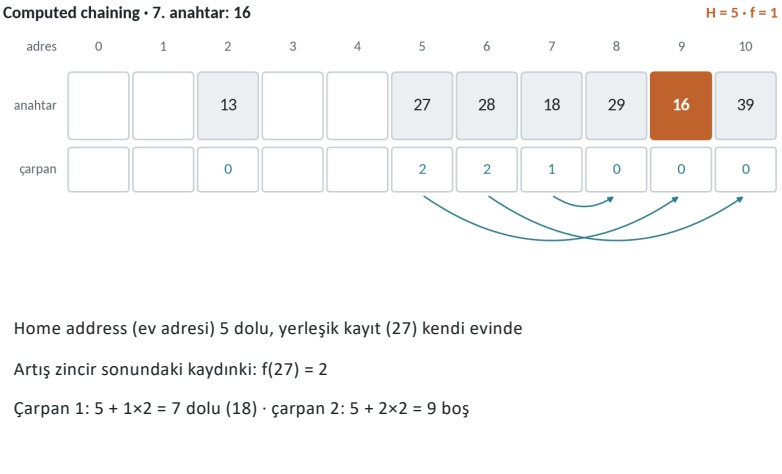
Computed chaining: 13



Altıncı adımda 13 anahtarı eklenmektedir. Home address (ev adresi) 2, artış değeri 1'dir. Ev adresi boş olduğundan kayıt doğrudan yerleşmekte ve pseudolink (sahte bağlantı) alanına sıfır yazılmaktadır. Sıfır, zincirin burada bittiğini göstermektedir. Bu adımdan sonra tabloda altı kayıt bulunmaktadır.

ADIM 7 / 9

Computed chaining: 16



Home address (ev adresi) 5 dolu, yerleşik kayıt (27) kendi evinde

Artış zincir sonundaki kaydınki: f(27) = 2

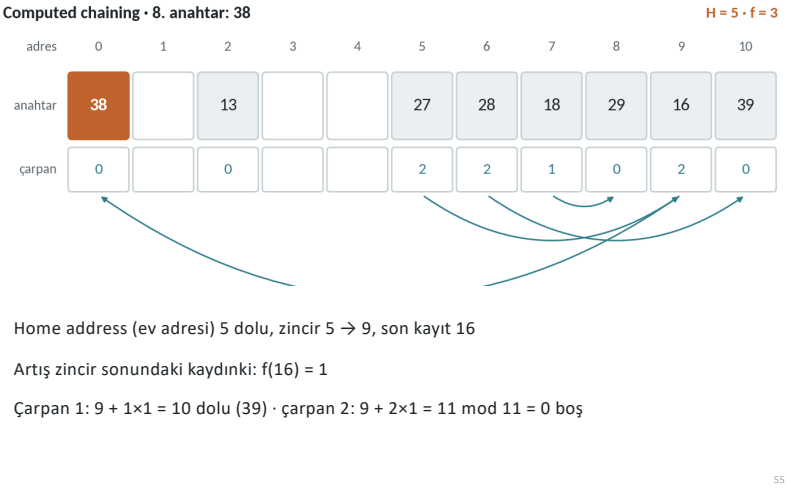
Çarpan 1: 5 + 1×2 = 7 dolu (18) · çarpan 2: 5 + 2×2 = 9 boş

54

Yedinci adımda 16 anahtarı eklenmektedir. Home address (ev adresi) 5'tir. Ev adresi doludur ve oradaki 27 kaydı kendi ev adresindedir; yeni kayıt zincirin sonuna eklenecektir. Zincirin son kaydı 5 adresindeki 27'dir ve boş yer 27'nin artış değeriyle aranır: f(27) = 2. Çarpan 1 ile 5 + 1×2 = 7 adresi 18 ile doludur, çarpan 2 ile 5 + 2×2 = 9 adresi boştur. 27'nin pseudolink (sahte bağlantı) alanına 2 çarpanı yazılmaktadır.

ADIM 8 / 9

Computed chaining: 38



Sekizinci adımda 38 anahtarı eklenmektedir. Home address (ev adresi) 5'tir. Ev adresindeki 27 kendi evindedir ve zinciri 5'ten 9'a uzanmaktadır. Zincirin son kaydı 9 adresindeki 16'dır; boş yer 16'nın artış değeriyle aranır: f(16) = 1. Çarpan 1 ile 9 + 1×1 = 10 adresi 39 ile doludur. Çarpan 2 ile 9 + 2×1 = 11 bulunur ve tablo boyuna göre mod alındığında 0 adresine varılır. Bu adres boş olduğundan 38 buraya yerleşir ve 16'nın pseudolink (sahte bağlantı) alanına 2 yazılır.

ADIM 9 / 9

Computed chaining: 53



Home address (ev adresi) 9: 16 kendi evinde değil

16 ve 38 zincirde ilerletilir, 53 evine yerleşir

56

Dokuzuncu adımda 53 anahtarı eklenmektedir. Home address (ev adresi) 9, artış değeri 4'dir. Ev adresi dokuzda bulunan on altı kaydı kendi ev adresinde değildir, yirmi yedinin zincirinden taşmıştır. Bu durumda yeni kayıt ev adresini almakta, on altı ve ondan sonra gelen otuz sekiz yirmi yedinin zinciri boyunca ilerletilmektedir. On altı sıfırıncı adrese, otuz sekiz birinci adrese yerleşmekte ve yirmi yedinin çarpanı ikiden üçe çıkmaktadır.

ÖZET

Dokuz anahtar

H(k) = k mod 11 f(k) = bölüm(k / 11) · 27, 18, 29, 28, 39, 13, 16, 38, 53

Anahtar	Ev adresi	Durum	İşlem	Yerleşti
27	5	boş	:	5
18	7	boş	:	7
29	7	18 evinde	7 + 1×f(18) = 8	8
28	6	boş	:	6
39	6	28 evinde	6 + 2×f(28) = 10	10
13	2	boş	:	2
16	5	27 evinde	5 + 2×f(27) = 9	9
38	5	zincir sonu 16	9 + 2×f(16) = 0	0
53	9	16 evinde değil	16 → 0, 38 → 1 taşındı	9

Örnekte dokuz anahtar on bir yuvalı bir tabloya yerleştirilmektedir. İlk sekiz anahtar zincir sonuna eklenerek yerleşir. 53 eklendiğinde home address (ev adresi) olan 9, kendi ev adresinde bulunmayan 16 kaydıyla doludur. Bu nedenle 16 ve ondan sonra gelen 38 zincir boyunca ilerletilir. 16, 27'nin zincirinde bir adım daha giderek 0 adresine, 38 ise 16'nın yeni konumundan bir adım giderek 1 adresine taşınır. 27'nin bağlantısı ikiden üçe çıkar.

DEĞERLENDIRME

Yöntemin sınırları

Probe sayısında kaydın zincirdeki sırası belirleyici

Bir kayıt silindiğinde sonraki bütün kayıtlar yeniden yerleştirilmeli

Ekleme ve silme daha fazla hesaplama, dolayısıyla daha uzun süre

- Buna karşılık kayıt okuma daha kısa sürüyor

Pseudolink (sahte bağlantı) alanının büyüklüğü belirleyici

Multiple chaining: tekli zincir birden fazla zincire parçalanır

- Ek bir hash function (hash fonksiyonu): g(k) = k mod R

Bir kayda ulaşmak için gereken probe sayısında kaydın zincirdeki sırası belirleyicidir. Bir kayıt silindiğinde, silinen kayıttan sonraki bütün kayıtları yeniden yerleştirmek gerekir. Ekleme ve silmede daha fazla hesaplamaya gerek duyulduğu için bu işlemler daha uzun sürer. Buna karşılık kayıt okuma daha kısa sürede gerçekleşir. Yöntemde pseudolink (sahte bağlantı) alanının büyüklüğü belirleyicidir.

ALIŞTIRMA 3 · SORU

Computed chaining

H(k) = k mod 11 f(k) = bölüm(k / 11), sıfırsa 1 · 14, 25, 36, 47, 3, 58, 24, 69, 35

Anahtar	Ev adresi	f(k)	Durum	Hesap	Yerleşti
14	?	?	?	?	?
25	?	?	?	?	?
36	?	?	?	?	?
47	?	?	?	?	?
3	?	?	?	?	?
58	?	?	?	?	?
24	?	?	?	?	?
69	?	?	?	?	?
35	?	?	?	?	?

Home address (ev adresindeki) kaydın kendi evinde olup olmadığına her adımda bakın

Toplam probe sayısını hesaplayıp coalesced hashing ile karşılaştırın

59

İkinci alıştırmada computed chaining tablosu doldurulacaktır. Her anahtar için home address (ev adresi), artış fonksiyonunun değeri, denenen adresler ve pseudolink (sahte bağlantıya) yazılan çarpan belirtilmelidir. Ev adresinde bulunan kaydın kendi evinde olup olmadığına her adımda bakılmalı, değilse hangi kayıtların ilerletildiği açıkça gösterilmelidir. Sonuçta toplam probe sayısı hesaplanmalı ve aynı anahtarlarla coalesced hashing'in yer maliyeti ile karşılaştırılmalıdır. Üç anahtarı için bölüm sıfır çıktığından artış değerinin bir alınacağı kuralı tabloya eklenmiştir. Bu kural olmadan zincir yerinde sayar ve boş yer bulunamaz.

ALIŞTIRMA 3 · ÇÖZÜM

Computed chaining

Anahtar	Ev	f(k)	İşlem	Yer	Probe
14	3	1	ev 3 boş	3	1
25	3	2	sona: 3 + 1×1 = 4	4	2
36	3	3	sona: 4 + 1×2 = 6	6	3
47	3	4	sona: 6 + 1×3 = 9	9	4
3	3	1*	sona: 9 + 1×4 = 2	10	5
58	3	5	sona: 2 + 3×1 = 5	0	6
24	2	2	2 boşaltıldı · 3: 9 + 3×4 = 10 · 58: 10 + 1×1 = 0	2	1
69	3	6	sona: 0 + 1×5 = 5	5	7
35	2	3	sona: 2 + 3×2 = 8	8	2

Toplam 31 probe, ortalama 3,44

* 3 için bölüm 0: artış 1 alınır, yoksa zincir ilerlemez

Çözümde her anahtar için home address (ev adresi) on bire bölümden kalan, artış değeri ise bölümün kendisidir. Anahtarların çoğu üç ile dört arasında kalan ev adreslerine düşmekte ve zincirler hızla uzamaktadır. Ev adresindeki kayıt kendi evindeyse yeni kayıt zincirin sonuna eklenmekte, değilse ev adresi boşaltılarak oradaki kayıt ve ardındakiler zincir boyunca ilerletilmektedir. Üç anahtarının on bire bölümü sıfır olduğundan artış değeri sıfır çıkmaktadır. Sıfır artışla zincir yerinde sayacağından, linear quotient (doğrusal bölüm) yöntemindeki gibi artış bir alınmaktadır. Alıştırmanın ifadesine bu kural eklenmiştir. Toplam probe sayısı otuz bir çıkmaktadır. Pseudolink (sahte bağlantı) alanlarında tutulan en büyük çarpan, alan için gereken bit sayısını belirlemektedir. Aynı anahtarlarla coalesced hashing (birleştirilmiş hashing) tablosu kurulduğunda link field (bağlantı alanının) tam adres tutması gerektiği ve dört bit gerektireceği görülmektedir.

7

Uygulama

Logical deletion (mantıksal silme) ve computed chaining C++ gerçeklemesi

C++ · 1

Logical deletion (mantıksal silme) · ekleme ve arama

```
constexpr int BOS = -1, SILINDI = -2;

int ekle(vector<int>& t, int k) {
    const int m = t.size();
    for (int i = 0; i < m; ++i) {
        int a = (k % m + i) % m;
        if (t[a] == BOS || t[a] == SILINDI) { t[a] = k; return a; }
    }
    return -1;
}

int ara(const vector<int>& t, int k, int& probe) {
    const int m = t.size();
    for (probe = 1; probe <= m; ++probe) {
        int a = (k % m + probe - 1) % m;
        if (t[a] == BOS) return -1;
        if (t[a] == k) return a;
    }
    return -1;
}
```

Ekleme iki tür yuvayı da kullanır, arama yalnızca BOŞ'ta durur

C++ · 2

Logical deletion (mantıksal silme) · silme

```
bool sil(vector<int>& t, int k) {
    int probe, a = ara(t, k, probe);
    if (a < 0) return false;
    t[a] = SILINDI;
    return true;
}
```

Kayıt önce aranır, bulunursa yuvası SİLİNDİ olarak işaretlenir

Yuva BOŞ yapılmaz: arama zinciri kopmasın

Avail list'teki silme işaretinin hash table (hash tablosundaki) karşılığı

ÇÖZÜMLÜ ÖRNEK

Silinen yerin iki işaretlenişi

```
vector<int> t(10, BOS);
for (int k : {35, 43, 70, 63, 30, 14, 23, 24, 34}) ekle(t, k);
vector<int> yanlis = t;
int p, a = ara(yanlis, 14, p);
yanlis[a] = BOS;
sil(t, 14);
for (int k : {23, 24, 34}) {
    int p1, p2;
    int a1 = ara(yanlis, k, p1), a2 = ara(t, k, p2);
    cout << k << "  BOS ile: " << a1 << " (" << p1 << ")"
         << "   SILINDI ile: " << a2 << " (" << p2 << ")\n";
}
cout << "56 eklendi: " << ekle(t, 56) << "\n";
```

ÇIKTI

```
23  BOS ile: -1 (4)   SILINDI ile: 7 (5)
24  BOS ile: -1 (3)   SILINDI ile: 8 (5)
34  BOS ile: -1 (3)   SILINDI ile: 9 (6)
56 eklendi: 6
```

BOŞ yapılınca 6'dan sonraki üç kayıt kayboluyor

SİLİNDİ ile hepsi bulunuyor

56 silinen yeri yeniden kullanıyor

C++ · 3

Computed chaining · zincir sonu

```
constexpr int BOS = -1;
struct Goz { int anahtar = BOS; int carpan = 0; };

int f(int k, int m) { int a = k / m; return a == 0 ? 1 : a; }

int sonraki(const vector<Goz>& t, int a) {
    const int m = t.size();
    return (a + t[a].carpan * f(t[a].anahtar, m)) % m;
}

void sona_ekle(vector<Goz>& t, int k) {
    const int m = t.size();
    int son = k % m;
    while (t[son].carpan != 0) son = sonraki(t, son);
    for (int j = 1; j < m; ++j) {
        int a = (son + j * f(t[son].anahtar, m)) % m;
        if (t[a].anahtar == BOS) {
            t[son].carpan = j;
            t[a] = {k, 0};
            return;
        }
    }
}
```

C++ · 4

Computed chaining · ekleme

```
void ekle(vector<Goz>& t, int k) {
    const int m = t.size();
    const int ev = k % m;
    if (t[ev].anahtar == BOS) { t[ev] = {k, 0}; return; }
    const int sahip = t[ev].anahtar % m;
    if (sahip == ev) { sona_ekle(t, k); return; }
    int once = sahip;
    while (sonraki(t, once) != ev) once = sonraki(t, once);
    vector<int> tasinan;
    for (int a = ev; ; ) {
        tasinan.push_back(t[a].anahtar);
        int s = t[a].carpan ? sonraki(t, a) : -1;
        t[a] = Goz{};
        if (s < 0) break;
        a = s;
    }
    t[once].carpan = 0;
    t[ev] = {k, 0};
    for (int x : tasinan) sona_ekle(t, x);
}
```

Home address (ev adresindeki) kayıt ve ardındakiler zincirden çıkarılıp yeniden sona eklenir

C++ · 5

Computed chaining · arama

```
int ara(const vector<Goz>& t, int k) {
    const int m = t.size();
    int a = k % m, probe = 1;
    if (t[a].anahtar == BOS || t[a].anahtar % m != a) return t[a].anahtar == k ? 1 : -1;
    while (t[a].anahtar != k) {
        if (t[a].carpan == 0) return -1;
        a = sonraki(t, a);
        ++probe;
    }
    return probe;
}
```

Home address (ev adresindeki) kayıt başka zincirdense aranan kayıt tabloda yoktur

Probe sayısı, kaydın zincirdeki sırasıdır

ÇÖZÜMLÜ ÖRNEK

Computed chaining, dokuz anahtar

```
vector<Goz> t(11);
const vector<int> anahtarlar = {27, 18, 29, 28, 39, 13, 16, 38, 53};
for (int k : anahtarlar) ekle(t, k);
for (int a = 0; a < 11; ++a)
    cout << a << ":" << t[a].anahtar << "/" << t[a].carpan
         << (a % 4 == 3 ? "\n" : "   ");
int toplam = 0;
for (int k : anahtarlar) toplam += ara(t, k);
cout << "\ntoplam " << toplam << ", ortalama " << toplam / 9.0;
```

ÇIKTI

```
0:16/1   1:38/0   2:13/0   3:-1/0
4:-1/0   5:27/3   6:28/2   7:18/1
8:29/0   9:53/0   10:39/0
toplam 14, ortalama 1.55556
```

Tablo elle izlemeyle aynı: 16 → 0, 38 → 1

27'nin çarpanı 2'den 3'e çıktı

14 probe, en büyük çarpan 3: iki bit